DATA 442/610: Deep Learning and Neural Networks
Assignment 3: RNNs and GNNs

Assigned: Oct 26, 2026; Due: Nov 8, 2026

Instructions

This assignment will be graded out of 100 points. It contains two problems. You are welcome to solve as many problems as you wish. Your score will be calculated based on the problems you turn in, up to a maximum of 100 points. **Any bonus questions will be scored and added on top of the 100-point grade total.**

- **Example:** You try Problem 1, scoring 95 points. You do not do Problem 2. Your grade will be 95/100.
- **Example:** You try Problem 1 and Problem 2, scoring 80 points on each. Your grade will be 100/100

Please submit each problem you attempt as a separate Jupyter notebook with labeled headings/subheadings for each question. This assignment will be graded based on demonstrated understanding of the core concepts. You do not need to achieve the best possible neural network accuracy. High marks will come from thoughtful documentation and explanation of *what* you are attempting, *how* you expect it to perform, and *why* you believe it will behave like it does. **Please comment your code**.

This assignment may require working out some intermediate mathematical steps. You are encouraged to show your work on any derivations underlying your code, and include this alongside your submission. You may submit any mathematical derivations as typeset equations in your Jupyter Notebooks. If you prefer not to type out that many equations, you may include a scanned PDF of handwritten calculations.

Note for Problem Set 3: For the problems in this assignment, one can train models to reasonable accuracy within 50 epochs, although you are welcome to train for longer if you like.

Problem 1 (100 points) - GNN revisits MNIST classification

In this problem, we'll go back to the MNIST dataset one more time, with a twist. Now, instead of classifying based on pixel intensity, you will build a model to classify digits from a graph representation of superpixels. A superpixel is a placeholder for a group of pixels. It can store, for example, the average intensity of a group of pixels. A superpixel may be connected to one or more superpixels based on aspects like position or its local feature value.

For more information, see Figure 2 of this paper.

Part 1: Data preprocessing (20 points)

The MNIST superpixels dataset is available at `torch_geometric.datasets.MNISTSuperpixels`. Load the dataset into your environment and separate it into training, validation, and testing datasets. Make sure to print out the size of each data subset.

Select a few random examples from your training dataset. Visualize them using the `networkx` package, using the attributes `sample.x` for superpixel intensity and `sample.pos` for superpixel position. You may find the functions `torch_geometric.utils.to_networkx` and `networkx.draw` useful. Use Figure 1 as a guide.

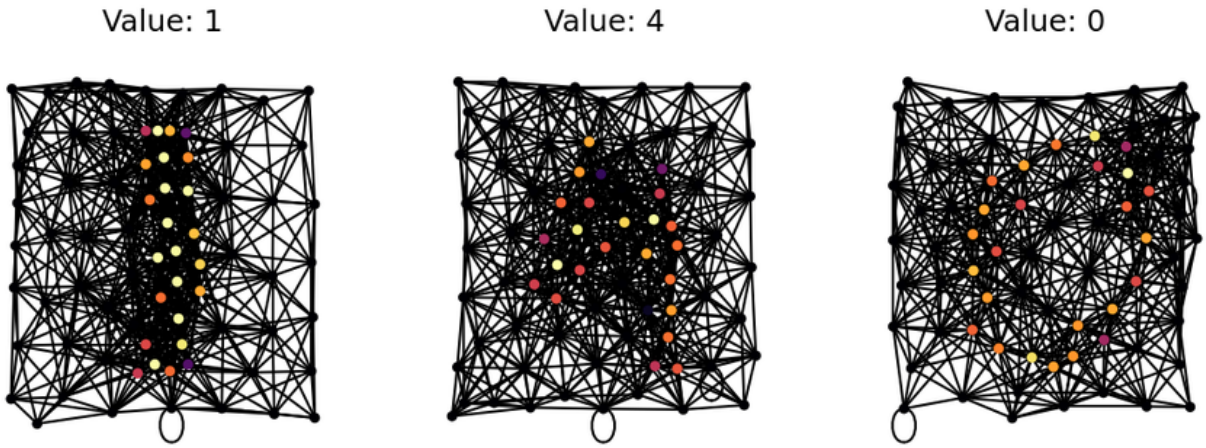


Figure 1: Sample visualization of MNIST superpixels data

Part 2: GCN implementation (30 points)

Implement a graph convolutional network. It should have at least two `torch_geometric.nn.GCNConv` layers, activation functions, some graph pooling operation, and a classifier head to map graph features into class probabilities. Explain your architecture choices, including network depth, width, pooling operations, activation functions, and others. You may use the in-class examples as a guide.

Train your model using the train/val datasets. Be sure to track loss and accuracy for both training and validation datasets. The optimizer, learning rate, training epochs, batch size, and other hyperparameters are entirely up to you. When finished, plot the training and validation curves and comment on your network's performance.

Part 3: Inference and analysis (10 points)

Run your trained graph convolutional network on the test dataset. Report the accuracy across all samples, and also print out the accuracy for each class. Does the model appear to show better or worse performance on individual classes? Why, or why not, do you think this occurs?

Plot a confusion matrix that shows the number of correctly and incorrectly classified test samples for each class. You may use tools such as `sklearn.metrics.ConfusionMatrixDisplay` if you like.

Part 4: Retraining with additional information (20 points)

The superpixel features x are not the only attributes available for each node. For this part, modify your training pipeline so that the network fits on the concatenation of the features x and the superpixel positions. That is, for a given data sample `sample`, the network should process `torch.cat([sample.x, sample.pos], dim=1)`. How you implement this modification is entirely up to you. Explain your implementation decision.

Retrain your model on this augmented data. Confirm that the input dimensionality is larger than the example in Part 2. Keep other hyperparameters the same. When finished, plot the loss curves, measure the overall and class-level accuracy on the test dataset, and visualize a new confusion matrix. Compare the performance of the "feature-only" model from Part 2 against this "position aware" model.

Part 5: Beyond GCN (20 points)

In this part, you are free to experiment with architectural choices. Try to design a model whose validation accuracy exceeds at least 80% within 50 epochs. You may consider alternative graph convolutions, greater width/depth, residual connections, other activations, different pooling, modified classifier heads, or any other ideas you think of. Explain any architectural changes you make and how you expect them to affect performance. You are welcome to run experiments as you develop your final architecture. If you do, comment on how performance aligned with your expectations.

When finished, run inference on the testing dataset, measure overall and class-wise accuracy, and plot a confusion matrix. Compare your model to the graph convolutional networks in Parts 2, 4. In addition, compare your model's performance to the CNN MNIST models we implemented in class (you are welcome to design and run your own if you like). Based on their performance, how useful is a graph structure for representing image data, and why do you think that? With that in mind, describe how you might adapt your graph neural networks to behave more like a traditional CNN?

Problem 2 (100 points) - RNN time-series forecasting

A Lorenz system is a canonical example of chaotic behavior that originated from studies of atmospheric convection. Trajectories, which represent airflow and temperature properties, are highly sensitive to initial conditions and may converge towards a butterfly-shaped pattern called a *strange attractor*. In this problem, you will use recurrent neural networks to forecast this chaotic system.

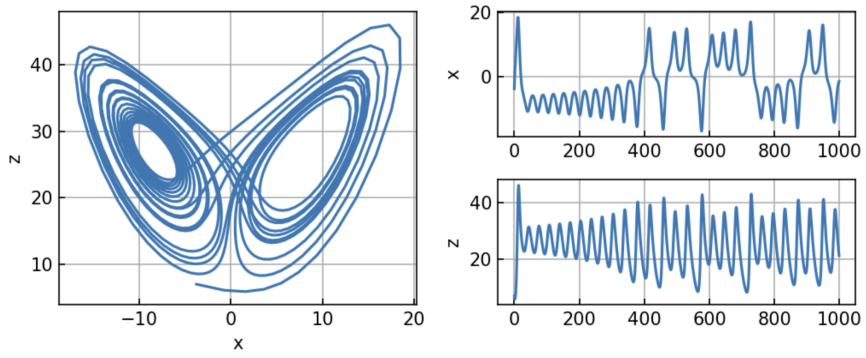


Figure 2: (*left*) 2D projection of Lorenz trajectory (*right*) Time series of each coordinate x, z

Part 1: Data preprocessing (20 points)

The code snippet below will simulate a Lorenz system from a random initial condition $[x_0, y_0, z_0]$ and return only the coordinates $x(t), z(t)$.

```
from scipy.integrate import solve_ivp

def lorenz(t, x, sigma=10., beta=8./3., rho=28):
    return [
        sigma * (x[1] - x[0]),
        x[0] * (rho - x[2]) - x[1],
        x[0] * x[1]  - beta * x[2]
    ]

def generate_lorenz(t_max=20, dt=0.02, x0=None, sigma=10., beta=8./3., rho=28):
    if x0 is None:
        x0 = -15 + 30 * np.random.uniform(size=3)
    t_eval = np.arange(0, t_max, dt)
    func = lambda t, x: lorenz(t, x, sigma=sigma, beta=beta, rho=rho)
    x = solve_ivp(func, (t_eval[0], t_eval[-1]), x0, t_eval=t_eval, rtol=1e-8,
    atol=1e-8).y.T
    return x[:, (0, 2)] # Return (x, z) coordinates
```

Question 1: Simulate and plot a Lorenz trajectory. Visualize it using Figure 2 as a guide.

Question 2: Simulate a Lorenz trajectory from an initial condition $[x_0, y_0, z_0]$ you choose. Simulate a second trajectory from a different initial condition $[x_0 + \delta_x, y_0 + \delta_y, z_0 + \delta_z]$ for some small δ. For each returned coordinate x, z, plot the trajectories beside each other. Comment on the behavior.

Question 3: Generate trajectories for training and validation. You may use different initial conditions, different parameter values, time resolutions, etc, by varying inputs to the `generate_lorenz` function above. Explain your decisions – how many sample trajectories, their conditions, etc. – and what challenges you expect when predicting.

Question 4: Write a Dataset class to hold your trajectory data. The `__getitem__` function should return an input sequence and a prediction target. You may use the examples from class as a guide.

Question 5: Visualize some samples from your dataset. Be sure to indicate what data will go into your model, and what data the model is trying to predict.

Part 2: Baseline models (20 points)

Choose at least two baseline models that are not RNNs. These can be multi-layer perceptrons, convolutional neural networks, or even non-deep learning models such as a moving average. These baseline models must be **non-trivially** different, meaning some variation beyond just changing depth, width, or hidden layer activations.

Explain why you chose your baseline models and how you expect them to perform. Train each models to forecast your trajectory datasets. Plot the training and validation curves and comment on the performance of each baseline relative to your prior expectations.

Part 3: RNN models (30 points)

Implement at least two RNN-based models. Explain why you chose those architectures and why you expect them to outperform your baseline models. Train them to forecast your trajectory datasets, plot the training and validation curves, and comment on the performance of each model relative to your prior expectations.

Part 4: Analysis (20 points)

Your "test" sample is the following:

```
test = generate_lorenz(t_max=15, dt=0.02, x0=[-8, 8, 28])
```

This command will produce a trajectory with 500 time points. The goal of this part is to predict the second half of this sample ($t = t_{251}, \ldots, t_{500}$) given the first half ($t = t_1, \ldots, t_{250}$).

For each of your baseline and RNN models, run an autonomous forecast of the test trajectory. After receiving initial input data and making an initial prediction t_{251}, each model should predict subsequent time points up to t_{500} by processing its own predictions.

Plot test predictions for each model. Comment on how well each model captures the "shape" of the target trajectory. Compute the test error for each model's predictions. For this forecasting problem, are there any potential pitfalls to using mean squared error or mean absolute error?

Part 5: More analysis (10 points)

The goal of this part is to measure an alternative error metric that captures how well each model captures the strange attractor of the Lorenz system. A good model will sweep out a similar set of points, while a bad model might orbit a circle or just decay to a constant prediction (see Figure 3).

To quantify this, we are going to estimate the *probability distribution* of the butterfly attractor and measure the Kullback-Leibler divergence between this and the predicted distribution of points.

$$D_{\mathrm{KL}}(p(x) \,\|\, q(x)) = \mathbb{E}_{x\sim p(x)}\left[\frac{\log p(x)}{\log q(x)}\right] = \sum_{x\in\mathcal{D}} p(x)\frac{\log p(x)}{\log q(x)} \geq 0$$

For a trajectory $\mathbf{x}(t)$, you can use a density estimation tool like `sklearn.neighbors.KernelDensity` to fit and estimate the probability distribution $p(\mathbf{x})$. Additionally, fit density estimators on the model trajectories to obtain distributions $q(\mathbf{x})$. Visualize the probability distributions for the ground truth $p(\mathbf{x})$ and your models $q(\mathbf{x})$, using the below figure as a guide.

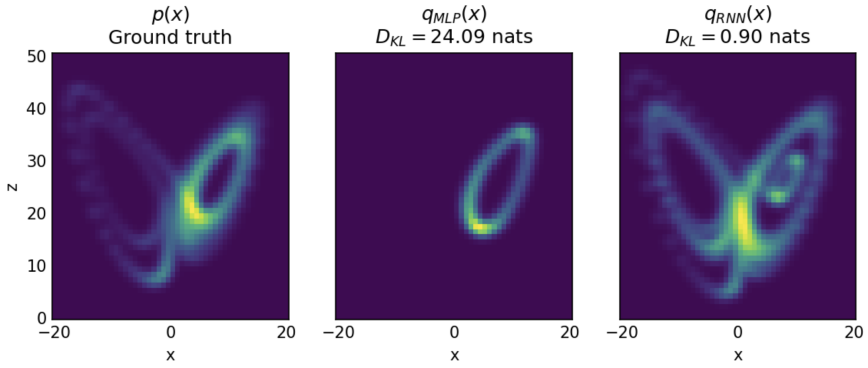


Figure 3: Example plots of estimated probability distributions.

Using these estimators, measure the Kullback-Leibler divergence using the formula above. Comment on the relative performance of each model. If D_{KL} were your error metric, how might you adjust your training procedure?

BONUS Part 6: Reservoir Computing (25 points)

A **reservoir computer** is a RNN framework that works remarkably well for some time-series problems including chaotic systems. The main ingredient of this framework is a large sparse hidden weight matrix whose weights are **frozen** during the training process. At each time step, the model predicts a new hidden state

$$\mathbf{h}(t+\Delta t) = \tanh\left[\, W_{\mathrm{in}}\,\mathbf{x}(t) + W_{\mathrm{res}}\,\mathbf{h}(t)\,\right] + \eta$$

Where W_{in} is a $R \times I$ fixed input matrix, W_{res} is a $R \times R$ fixed reservoir matrix, and η is some optional small-magnitude noise. The predictions are given by

$$\mathbf{x}(t+\Delta t) = W_{\mathrm{out}}\,\mathbf{h}(t+\Delta t)$$

Where W_{out} is a $O \times R$ trainable output matrix. To fit a reservoir computer, one generates hidden states using the ground truth trajectory and then fits W_{out} using least squares or L2-regularized least squares regression. This is much cheaper and faster than training an RNN over many epochs.

Implement a reservoir computer, fit it on the first half of the test trajectory in Part 4, and forecast the second half of that trajectory. You may use the in-class template as a starting point. Compare the error and Kullback-Leibler divergence to your baseline and RNN models. We recommend a large ($R > 500$) reservoir with sparse entries ($< 1\%$) rescaled to have a maximum eigenvalue $|\lambda_{\max}| < 1$.